COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	8/05/26	**DATE SUBMITTED:**	
NAME:	Gabriel Dwyane D. Aridido n	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO 5.1 **Hands-on Activity 5.1 Queues**

1. PROCEDURE OUTPUT

ILO A: Queue using STL C++ Library

mainA.cpp | mainB.cpp

ILOA > mainA.cpp > display(std::queue<char>)

```
1   #include<iostream>
2   #include<queue>
3
4   //display
5       void display(std::queue<char> copyQ) {
6
7           //loop until empty
8           std::cout << "myQ: ";
9           while(!copyQ.empty()) {
10              //display front
11              std::cout << copyQ.front() << " ";
12              //dq the front
13              copyQ.pop();
14
15          }
16          std::cout << std::endl;
17
18      }
19
20  int main(){
21
22      //create an object:
23      std::queue<char> myQ;
24
25      //use the enqueue
```

ILOA > mainA.cpp > main()

```cpp
    void display(std::queue<char> copyQ) {
    }

int main(){

    //create an object:
    std::queue<char> myQ;

    //use the enqueue
    myQ.push('G');
    myQ.push('A');
    myQ.push('B');

    std::cout << "Current Front is: " << myQ.front() << std::endl;
    std::cout << "Current Back is: " << myQ.back() << std::endl;
    display(myQ);

    //dequeue
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);

    //empty checker
    std::cout << "Is the queue empty? " << myQ.empty() << std::endl;



    return 0;

}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\TIPQC\Documents\CPE21S2 ARIDIDON\HOA5p1\ILOA> cd "c:\Users\TIPQC\Documents\CPE21S2 ARIDIDON\HOA5p1\ILOA\" ; if ($?) { g++ mainA.cpp -o mainA } ; if ($?) { .\mainA }
Current Front is: G
Current Back is: B
myQ: G A B
myQ: A B
myQ: B
myQ:
Is the queue empty? 1
```

ILOB: Queue using linked lists

ILOB > C queueLL.h > ...

```h
#ifndef QUEUE_LIST_H
#define QUEUE_LIST_H


// create a class node
template<typename T>
class qNode{
    public:
        T data;
        qNode* next;
};

// new nodecreator
template <typename T>
qNode<T>* new_node(T newData) {
    //allocate a space for the new node
    qNode<T>* newNode = new qNode<T>;

    //stpre mewdata tp the newnode
    newNode -> data = newData;

    //point newNode next to null
    newNode -> next = nullptr;

    return newNode;

}

```

```
28
29    // enqueue
30    template <typename T>
31    void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr) {
32
33        //create new node
34        qNode<T>* newNode = new_node(newData);
35
36        //check if q is empty
37        if((*frontPtr) == nullptr && (*backPtr) == nullptr) {
38            (*frontPtr) = newNode;
39            (*backPtr) = newNode;
40            return;
41        }
42
43
44        //if not, insert at the back
45        (*backPtr) -> next = newNode;
46        (*backPtr) = newNode;
47    }
48
49
```

ILOB > mainB.cpp > main()

```
1  #include<iostream>
2  #include "queueLL.h"
3
4
5  int main() {
6      qNode<char>* front = nullptr;
7      qNode<char>* back = nullptr;
8
9      enqueue('G', &front, &back);
10     std::cout << "front: " << front->data << " " << "back: " <<back->data << std::endl;
11     enqueue('A', &front, &back);
12     std::cout << "front: " << front->data << " " << "back: " <<back->data << std::endl;
13     enqueue('B', &front, &back);
14     std::cout << "front: " << front->data << " " << "back: " <<back->data << std::endl;
15
16     return 0;
17 }
```

```
PS C:\Users\TIPQC\Do
front: G back: G
front: G back: A
front: G back: B
```

```cpp
//dequeue
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
    // check if the q is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout << "The queue is empty"<<std::endl;
        return;
    }


    //create a temp variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;


    //assign the current front to deleteTemp
    deleteTemp = (*frontPtr);


    //check if the q has only 1 item
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        //means only 1 item is inside the q
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;

    }
    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;
    // delete node
    delete deleteTemp;

}
```

```cpp
#include<iostream>
#include "queueLL.h"


int main() {
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout << "Testing the enqueue operator: \n";
    enqueue('G', &front, &back);
    std::cout << "front: " << front->data << " " << "back: " <<back->data << std::endl;
    enqueue('A', &front, &back);
    std::cout << "front: " << front->data << " " << "back: " <<back->data << std::endl;
    enqueue('B', &front, &back);
    std::cout << "front: " << front->data << " " << "back: " <<back->data << std::endl;

    std::cout << "Testing the dequeue operator: \n";
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " " << "back: " <<back->data << std::endl;
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " " << "back: " <<back->data << std::endl;
    dequeue(&front, &back);
    dequeue(&front, &back);
```

```
front: G back: G
front: G back: A
front: G back: B
Testing the dequeue operator:
front: A back: B
front: B back: B
The queue is empty
```

```cpp
//display all
template<typename T>
void qLLTraverse(qNode<T>** front) {
    if((*front) == nullptr) {
        std::cout << "Queue is empty" <<std::endl;
        return;
    }

    while((*front) !=nullptr){
        std::cout << (*front)->data;
        (*front) = (*front)->next;

    }
    std::cout << std::endl;
}
```

```
  std::cout << "Testing the qllTraverse operator: \n";
  qLLTraverse(&front);
```

```
front: G back: G
front: G back: A
front: G back: B
Testing the dequeue operator:
front: A back: B
front: B back: B
The queue is empty
Testing the qllTraverse operator:
Queue is empty
```

```
//isEmpty
template<typename T>
bool isEmpty(qNode<T> **front, qNode<T> **back) {
    if((*front) == nullptr && (*back) == nullptr)
        return true;

        return false;


}



#endif
```

ILO B.2 Array-Based Implementation

ILOB2 > C queueAR.h > Full()

```
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>


//array-based circ q
template<typename T>
class queueArr{
    private:
        //ptr to dynamically allocate array
        T* q_array;
        //max no of element
        size_t q_capacity;
        //current no of elements
        size_t q_size;
        //index of the front element
        int q_front;
        //index of back element
        int q_back;


    public:
        //constructor
        queueArr(size_t cap = 10);

        //copy constructor
        queueArr(const queueArr& other);

        //copy assignment operator
        queueArr& operator=(const queueArr&other);

        //destructor
        ~queueArr();


        //q prototype operations:
        bool Empty();
        bool Full();
        size_t Size();
        void Clear();
        T Front();
        T Back();
        void Enqueue(T value);
        T Dequeue();

};
```

```cpp
//constructor
template <typename T>
queueArr<T>::queueArr(size_t cap) {
    q_capacity = cap;
    q_size = 0;
    q_front = 0;
    q_back = -1;

    q_array = new T[q_capacity];
}

//copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other) {
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    //create new array
    q_array = new T[q_capacity];

    // copy the elements
    for(size_t i = 0; i < q_capacity; i++)
    {
        q_array[i] = other.q_array[i];
    }
}
```

```cpp
//copy assignment operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
    // prevent self assignment
    if(this == &other)
    {
        return *this;
    }


    // delete old memory
    delete[] q_array;


    // copy data
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;


    // allocate new memory
    q_array = new T[q_capacity];


    // copy elements
    for(size_t i = 0; i < q_capacity; i++)
    {
        q_array[i] = other.q_array[i];
    }


    return *this;
}

//destructor
template <typename T>
queueArr<T>::~queueArr() {
    delete[] q_array;
}
```

Constructor initializes the queue by setting the capacity, size, front, and back values, then dynamically allocating the array. The initial q_size is 0 because no elements have been added yet, while q_front is 0 and q_back is -1 to indicate that the queue is empty.

Copy Constructor

The copy constructor creates a new queue with the same capacity, size, front, and back values as the original queue, then creates a separate array and copies all the elements. This is why the copied queue produces the same output as the original when calling functions like Front() or Back(). However, the two queues have separate memory, so changing one does not directly affect the other.

Copy Assignment Operator - used when an already existing queue is assigned another queue. It first deletes the old array, copies the information from the other queue, creates a new array, and copies its elements. Therefore, after q2 = q1, both queues contain the same values, so functions such as Front(), Back(), and Size() give the same results for both queues.

Destructor uses delete[] to release the dynamically allocated array when the queue object is destroyed. It normally produces no output because its purpose is memory management rather than displaying queue values.

```cpp
//Empty()
template <typename T>
bool queueArr<T>::Empty(){
   return q_size == 0;

}

//Full()
template <typename T>
bool queueArr<T>::Full(){
    return q_size == q_capacity;

}


//find current size of array
template <typename T>
size_t queueArr<T>::Size() {
    return q_size;
}

//front()
template <typename T>
T queueArr<T>::Front(){
    //check if the q is empty
    if(Empty()) {
        std::cout<<"Queue is Empty\n";
        return T();
    }
    //return thefront
    return q_array[q_front];


}

//back()
template <typename T>
T queueArr<T>::Back(){
    //check if the q is empty
    if(Empty()) {
        std::cout<<"Queue is Empty\n";
        return T();
    }
    //return the back
    return q_array[q_back];


}
```

```cpp
//clear()
template <typename T>
void queueArr<T>::Clear(){

    //reset q_size, front, back
    q_size = 0;
    q_front = 0;
    q_back = -1;


}

//enqueue
template <typename T>
void queueArr<T>::Enqueue(T value){
    //check if the q is full
    if(Full()) {
        std::cout<<"Queue is Full\n";
        return;
    }

    //move q_back circularly
    q_back = (q_back + 1) % q_capacity;

    //store value to the back
    q_array[q_back] = value;

    // increment the q_size
    q_size++;

}


//dequeue
template <typename T>
T queueArr<T>::Dequeue(){
    //check if the q is full
    if(Empty()) {
        std::cout<<"Queue is Empty\n";
        return T();
    }

    //create a temp variable to store front
    T temp = q_array[q_front];

    //move  q_front circularly
    q_front = (q_front + 1) % q_capacity;

    // decrement the q_size
    q_size--;

    //reset the indexes if the q become empty
    if (Empty()) Clear();

    // return the temp variable
    return temp;
}
```

The **Empty()** function checks whether q_size is equal to 0. If the queue has no elements, it returns true; otherwise, it returns false. The **Full()** function checks whether q_size has reached q_capacity. The **Size()** function simply returns the current value of q_size. Every successful **Enqueue()** increases the size by one, while every successful **Dequeue()** decreases it by one. The **Clear()** function resets q_size to 0, q_front to 0, and q_back to -1, making the array empty. The **Back()** function returns the element at q_back, which represents the most recently inserted element.

Testing on main + output

```cpp
#include <iostream>
#include "queueAR.h"

int main(){

    //creating an object
    queueArr<int> q(5);

    std::cout<<"Is array empty? " << q.Empty() << std::endl;
    q.Enqueue(1);
    std::cout<< "Front: " << q.Front() << std::endl;
    std::cout<< "Back: " << q.Back() << std::endl;
    std::cout<< "Size: " << q.Size() << std::endl;


    q.Enqueue(2);
    q.Enqueue(3);
    std::cout<< "Front: " << q.Front() << std::endl;
    std::cout<< "Back: " << q.Back() << std::endl;
    std::cout<< "Size: " << q.Size() << std::endl;



    q.Enqueue(4);
    std::cout<< "Front: " << q.Front() << std::endl;
    std::cout<< "Back: " << q.Back() << std::endl;
    std::cout<< "Size: " << q.Size() << std::endl;

    q.Enqueue(5);
    std::cout<< "Front: " << q.Front() << std::endl;
    std::cout<< "Back: " << q.Back() << std::endl;
    std::cout<< "Size: " << q.Size() << std::endl;

    q.Enqueue(6);

    std::cout<< "Removed: " << q.Dequeue() << std::endl;


    std::cout<<"Is array Full? " << q.Full() << std::endl;

    std::cout<<"---Testing clear---" << std::endl;
    q.Clear();
    std::cout<<"Is array Empty? " << q.Empty() << std::endl;



    return 0;
}
```

```
Is array empty? 1
Front: 1
Back: 1
Size: 1
Front: 1
Back: 3
Size: 3
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
Queue is Full
Removed: 1
Is array Full? 0
---Testing clear---
Is array Empty? 1
```

The output shows the behavior of the array-based circular queue following the FIFO (First In, First Out) principle. At the start, Empty() returns 1 because the queue has no elements and q_size is 0. After inserting values 1 to 5 using Enqueue(), the front remains 1 because it is the first element added, while the back changes to the newest inserted value (1, 3, 4, and 5). The size also increases after every successful insertion until it reaches the capacity of 5. When q.Enqueue(6) is called, the queue is already full, so the program displays "Queue is Full" and does not add 6. The next operation, Dequeue(), removes 1 because it is the oldest element in the queue, reducing the size and making the queue no longer full, which is why Full() returns 0. Finally, calling Clear() resets the queue by setting the size back to 0 and resetting the indexes, causing Empty() to return 1 because the queue is now considered

empty.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation
Array Implementation

```
//job class
class Job{
    private:
        int id;
        std::string user;
        int pages;

    public:
        Job(int i=0, std::string user = "", int pages = 0){
            id = i;
            this->user = user;
            this->pages = pages;
        }

        void Display(){
            std::cout << "Job ID: " << id << " | User: " << user << " | Pages: " << pages << std::endl;
        }

};
```

```
//printer class
class Printer{
    private:
        queueArr<Job> q;
    public:
        Printer(size_t capacity = 10) : q(capacity) {}
        void Addjob(Job newJob){
            q.Enqueue(newJob);
        }

        void PrintJob(){
            if(q.Empty()) {
                std::cout<<"No jobs\n";
                return;
            }

        Job newJob = q.Dequeue();
        newJob.Display();
    }
        void PrintAll(){
            while(!q.Empty()){
                PrintJob();
            }
        }

};
```

The Job class is used to represent each printing request made by a user. It stores the important details of a job, which are the job ID, the user's name, and the number of pages that need to be printed. The constructor assigns the values when a new job is created, while the Display() function is used to show the job information when the printer

processes it. The Printer class is responsible for managing the printing requests. It uses the queueArr<Job> that I created earlier to store all the pending jobs. The AddJob() function adds new print requests to the queue, while the PrintJob() function removes and processes the oldest job in the queue. The PrintAll() function continues printing until there are no more jobs left. The reason it uses a queue is because a real printer follows a first come, first served system. The first person who sends a print request should be the first person whose document gets printed.

Why Array and not linked list?

I decided to use an array implementation because I am more familiar and comfortable working with arrays compared to linked lists. Since this activity only requires simulating a printer queue with a limited number of jobs, using an array is already enough for the problem.

Another reason I chose arrays is because they are easier to visualize and manage. In my implementation, I used a circular queue where the front and back indexes move instead of shifting all the elements whenever a job is removed. This makes adding and removing jobs more efficient while still following the FIFO principle.

Although linked lists have the advantage of having a flexible size, I chose arrays because they are simpler for me to implement and understand, and they still satisfy the requirements of the activity.

Maincpp + Output

queueAR.h ILOC | mainC.cpp | queueAR.h ILOB2

ILOC > mainC.cpp > main()

```cpp
#include<iostream>
#include "queueAR.h"

int main(){
    //create object
    Printer officePrinter(3);


    std::cout << "=== Adding Print Jobs to Queue ===\n";
    officePrinter.Addjob(Job(101, "Parker ", 15));
    std::cout << "Parker submitted a printing job\n";
    officePrinter.Addjob(Job(102, "Reily", 23));
     std::cout << "Reily submitted a printing job\n";
    officePrinter.Addjob(Job(103, "Watts", 5));
     std::cout << "Watts submitted a printing job\n";

    std::cout << "\n=== Processing First 3 Jobs ===\n";
    officePrinter.PrintJob();
    officePrinter.PrintJob();
    officePrinter.PrintJob();


    std::cout << "\n=== Adding More Jobs ===\n";
    officePrinter.Addjob(Job(104, "White", 15));
    std::cout << "White submitted a printing job\n";
    officePrinter.Addjob(Job(105, "Castle", 3));
    std::cout << "Castle submitted a printing job\n";
    officePrinter.Addjob(Job(106, "Banner", 10));
    std::cout << "Banner submitted a printing job\n";
    officePrinter.Addjob(Job(107, "Doe", 20));

    officePrinter.PrintAll();

    return 0;
}
```

Problems | Output | Debug Console | Terminal | Ports

```
PS D:\GABRIEL FILES\A\CPE\2nd Year\1st Sem\CPE 010 - DSA\Prelim_HOA 5
 ; if ($?) { g++ mainC.cpp -o mainC } ; if ($?) { .\mainC }
=== Adding Print Jobs to Queue ===
Parker submitted a printing job
Reily submitted a printing job
Watts submitted a printing job

=== Processing First 3 Jobs ===
Job ID: 101 | User: Parker  | Pages: 15
Job ID: 102 | User: Reily | Pages: 23
Job ID: 103 | User: Watts | Pages: 5

=== Adding More Jobs ===
White submitted a printing job
Castle submitted a printing job
Banner submitted a printing job
Queue is Full
Doe submitted a printing job
Job ID: 104 | User: White | Pages: 15
Job ID: 105 | User: Castle | Pages: 3
Job ID: 106 | User: Banner | Pages: 10
```

The output happened this way because the program follows the behavior of a real printer queue. Jobs are stored in the order they are submitted, and the printer processes them one by one based on who entered the queue first.

The array capacity also affects the output because once the queue reaches its limit, any additional jobs cannot be added until there is available space. This simulation shows how queues can be used to manage tasks that need to be completed in a specific order.

Conclusion

In this activity, I learned how queues work and how they can be applied to real-world situations such as managing printer requests. I gained a better understanding of the FIFO (First In, First Out) principle, circular queue implementation using arrays, and how different classes like Job and Printer can work together to organize and process data. The procedure helped me understand the importance of checking queue conditions, managing capacity, and applying data structures in practical scenarios. The supplementary activity made the concept easier to understand because it showed how queues are used in systems where tasks need to be handled in order. Overall, I think I performed well in this activity since I was able to create and test a working printer simulation, but I still need to improve my knowledge of other implementations like linked lists and continue practicing with more complex programs involving multiple classes and dynamic memory management.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Guide for making the big 3 after constructor**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI as a guide while working on the activity, mainly to help me understand errors, review my code structure, and clarify concepts that I found difficult. However, I still made sure to analyze and understand the code instead of relying completely on the suggestions.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.